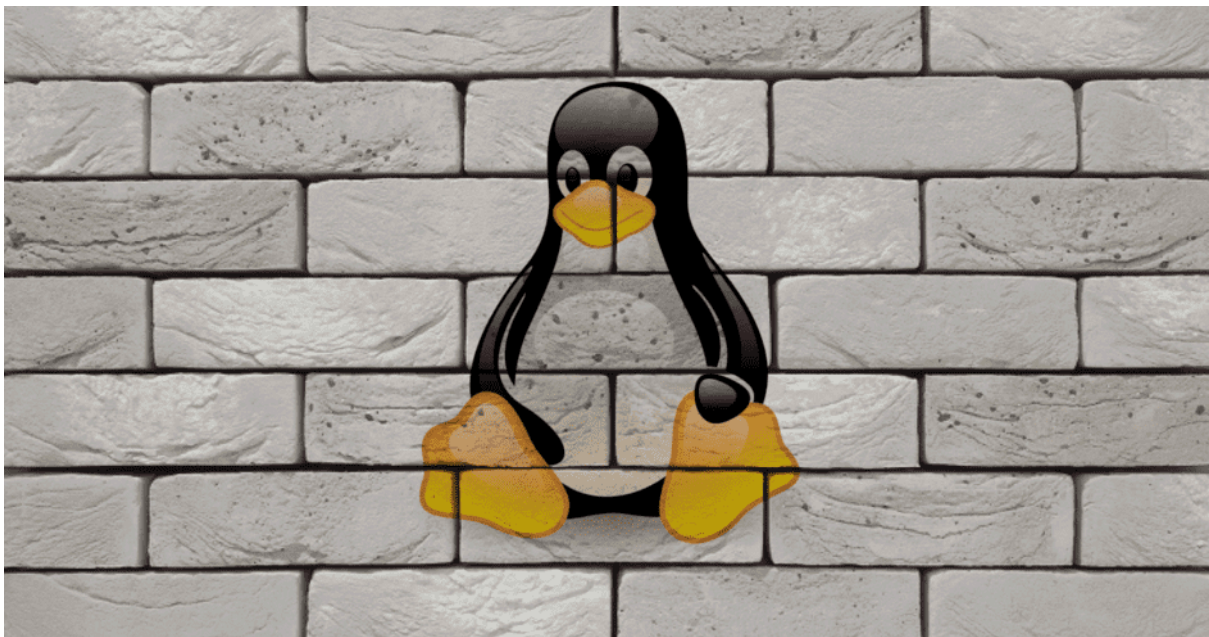




**Instituto Superior
de Engenharia**

Politécnico de Coimbra

Sistemas Operativos - 2024/25



João Rosa - 2022131973
Gonçalo Silva - 2022148262

Índice

Índice	2
1. Introdução	3
2. Estrutura e arquitetura do sistema	4
2.1 Structs.h	4
3. Implementação	7
3.1 Feed	7
3.1.1 Inicialização dos Recursos	7
3.1.2 Login do Cliente	8
3.1.3 Gestão de mensagens recebidas	9
3.1.4 Comandos do Utilizador	10
3.2 Manager	11
3.2.1 Inicialização dos recursos	11
3.2.2 Select para teclado e receção de named pipes	12
3.2.3 Comandos do Teclado	12
3.2.4 Comandos do Cliente	17
3.2.5 Guardar e Abrir Ficheiro	23
3.3 Temporizador das Mensagens Persistentes	25
4. Makefile	26
4.1 Estrutura	26
4.2 Regras	26
4.3 Compilação	27
5. Conclusão	28

1. Introdução

Este trabalho prático teve como objetivo implementar uma plataforma de envio e recepção de mensagens curtas, organizada por tópicos, em linguagem C, para sistemas UNIX. A solução desenvolvida foi baseada nos requisitos especificados no enunciado, com ênfase no uso correto das chamadas de sistema e dos mecanismos estudados na unidade curricular.

Neste trabalho tivemos a ideia de tentar implementar toda a matéria da UC para comprovar o nosso alto nível de conforto com a mesma.

2. Estrutura e arquitetura do sistema

A plataforma foi implementada com dois programas principais e um ficheiro .h com as estruturas e a definição das funções:

2.1 Structs.h

- Define as constantes pedidas pelo enunciado.

```
#define MAX_USERS 10
#define MAX_TOPICS 20
#define MAX_MSG_PERSISTENTE 5
#define MAX_MSG_SIZE 300
#define MAX_MSG_TO_FEED_SIZE 500
#define MAX_TOPIC_SIZE 20
```

- Define as estruturas de dados
 - Estrutura MSG: estrutura que é enviada para o servidor (Manager.c) sempre que algum cliente realizar alguma operação. Esta estrutura tem o utilizador que envia, nome do tópico, um inteiro para definir se é uma mensagem persistente e o texto da mensagem:

```
typedef struct {
    User usr;
    char msg[MAX_MSG_SIZE];
    char topico[MAX_TOPIC_SIZE];
    int dur;
} MSG;
```

- Estrutura User -> nome e pid de cada utilizador:

```
typedef struct {
    int pid;
    char username[30];
} User;
```

- Estrutura MsgToFeed: Estrutura enviada para o cliente sempre que o servidor precisa de se comunicar com este. Esta estrutura apresenta apenas a mensagem e um mutex que será explicado mais à frente:

```
typedef struct {
    char mensagem[MAX_MSG_TO_FEED_SIZE];
    pthread_mutex_t *m;
} MsgToFeed;
```

- Estrutura Topic: Maior estrutura do código que apresenta todos os dados necessários para um tópico. Esta estrutura apresenta um nome, um array de utilizadores, um array de mensagens (só as mensagens persistentes entram neste array), dois inteiros, um para o número de utilizadores e outro para o número de mensagens persistentes no tópico.

Por fim apresenta um inteiro que serve de Boolean que diz se o tópico está bloqueado ou não (1 -> sim, 0 -> não):

```
typedef struct {
    char nome[MAX_TOPIC_SIZE];
    User users[MAX_USERS];
    MSG msgPer[MAX_MSG_PERSISTENTE];
    int numUsers, numMsgs, block;
    pthread_mutex_t* m;
} Topic;
```

- Define as funções todas que serão explicadas mais à frente:

```
void removeTopic(Topic* topic, Topic topics[]);
void exit_signal(User users[]);
int login(User *new_user, User users[]);
void loginIn(MSG msg);
void printUsers(User users[]);
int msgManager(MSG msg, Topic topics[]);
void printTopics(Topic topics[]);
int subscribe(MSG *msg, Topic topics[]);
int addPersistentMsg(MSG msg, Topic topics[]);
int unsubscribe(MSG *msg, Topic topics[]);
void lock(Topic topics[], char topic[]);
void unlock(Topic topics[], char topic[]);
void save_persistent_messages(const char* filename, Topic topics[]);
void load_persistent_messages(const char* filename, Topic topics[]);
void showTopic(Topic topics[], char topic[]);
void removeTopic(Topic *topic, Topic topics[]);
void removeUser(User *usr, Topic topics[], User users[]);
void userExitHandler(int signum, siginfo_t *info, void *secret);
void* TimerPersistentMsg(void* dados);
void *recebePipes(void* msgFeed);
```

3. Implementação

3.1 Feed

3.1.1 Inicialização dos Recursos

- O programa verifica os argumentos de entrada do teclado.

```
if(argc!=2){  
    printf("Erro de Sintaxe: <username>\n");  
    return -1;  
}
```

- O programa inicializa um *sigaction* para apanhar o sinal *SIGUSR1*, garantindo que o *named pipe* do cliente seja removido adequadamente caso o servidor termine (A parte do servidor vai ser mostrada em baixo).

```
struct sigaction sa;  
sa.sa_sigaction = userExitHandler;  
sa.sa_flags = SA_SIGINFO;  
if(sigaction(SIGUSR1, &sa, NULL) == -1){  
    perror("Erro ao configurar SIGUSR1");  
    exit(1);  
}
```

```
void userExitHandler(int signum, siginfo_t *info, void *secret){
    printf("Server closed!\n");
    unlink(FEED_FIFO_FINAL);
    exit(1);
}
```

- O programa cria um *mutex* e uma *thread* para gerenciar a leitura de mensagens recebidas (Função mostrada mais em baixo).

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
pthread_t t;
msgFeed.m = &mutex;

if(pthread_create(&t, NULL, &recebePipes, &msgFeed) != 0){
    perror("Erro ao criar thread\n");
    exit(-1);
}
```

3.1.2 Login do Cliente

- A função *LoginIn* implementa a lógica do login do cliente. Esta função:
 - Envia o nome do utilizador e o *PID* do cliente para o *named pipe* do servidor.

```
void loginIn(MSG msg){
    int fd_user_envia, fd_user_recebe, x, size2;

    fd_user_envia = open(MANAGER_FIFO, O_WRONLY);
    if(fd_user_envia == -1){
        printf("Erro ao abrir o servidor");
        unlink(FEED_FIFO_FINAL);
        exit(1);
    }

    strcpy(msg.msg, "");
    strcpy(msg.topico, "");

    write(fd_user_envia, &msg, sizeof(msg));
}
```

- Aguarda a resposta indicando sucesso ou falha (como usuário duplicado ou limite atingido).

```
fd_user_recebe = open(FEED_FIFO_FINAL, O_RDONLY);
if(fd_user_recebe == -1){
    printf("Erro ao abrir o cliente");
    exit(1);
}

size2 = read(fd_user_recebe, &x, sizeof(x));
if(size2 >= 0){
    if(x == 0){
        printf("\nJa existe um utilizador com esse username ou limite de utilizadores atingido!\n");
        close(fd_user_recebe);
        unlink(FEED_FIFO_FINAL);
        exit(1);
    }
    else
        printf("Logado com sucesso!\nBem-vindo!\n");
}
```

3.1.3 Gestão de mensagens recebidas

- Foi utilizada uma *thread* para receber e mostrar todas as mensagens recebidas do servidor, ou seja, este programa simplesmente mostra informação vinda do servidor.
- É utilizado o mutex sempre que a thread executa um printf para evitar conflitos com os comandos do utilizador.

```
void *recebePipes(void* msgFeed){
    MsgToFeed *msg = (MsgToFeed*) msgFeed;
    int size;
    int fd_recebe = open(FEED_FIFO_FINAL, O_RDONLY);
    if(fd_recebe == -1){
        perror("Erro ao abrir cliente");
        unlink(FEED_FIFO_FINAL);
        exit(1);
    }
    while(1){
        size = read(fd_recebe, &msg->mensagem, sizeof(msg->mensagem));
        if(size > 0){
            pthread_mutex_lock(msg->m);
            printf("%s", msg->mensagem);
            pthread_mutex_unlock(msg->m);
            if(strcmp(msg->mensagem, "Foste removido!\n") == 0){
                close(fd_recebe);
                unlink(FEED_FIFO_FINAL);
                exit(-1);
            }
        }
    }
    close(fd_recebe);
    return NULL;
}
```

3.1.4 Comandos do Utilizador

- No main, encontra-se um ciclo que está constantemente a receber comandos vindo do teclado. Compara o que veio do teclado com uma lista de comandos e, se algum comando for válido, envia o mesmo por *named pipe* para o servidor.

```
while(1){
    strcpy(msg.usr.username, argv[1]);
    msg.usr.pid = getpid();
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "topics") == 0) {
        strcpy(msg.msg, "topics");
        fd_envia = open(MANAGER_FIFO, O_WRONLY);
        if(fd_envia == -1){
            perror("Erro ao abrir o cliente");
            exit(1);
        }
        printf("%s", msg.topico);
        write(fd_envia, &msg, sizeof(msg));
    }
    else if (sscanf(input, "msg %s %d %[^\\n]", msg.topico, &msg.dur, msg.msg) == 3) {
        fd_envia = open(MANAGER_FIFO, O_WRONLY);
        if(fd_envia == -1){
            perror("Erro ao abrir o cliente");
            exit(1);
        }
        write(fd_envia, &msg, sizeof(msg));
        memset(&msg, 0, sizeof(msg));
    }
    else if (sscanf(input, "subscribe %s", msg.topico) == 1) {
        strcpy(msg.msg, "subscribe");
        fd_envia = open(MANAGER_FIFO, O_WRONLY);
        if(fd_envia == -1){
            perror("Erro ao abrir o cliente");
            exit(1);
        }
        write(fd_envia, &msg, sizeof(msg));
    }
    else if (sscanf(input, "unsubscribe %s", msg.topico) == 1) {
        strcpy(msg.msg, "unsubscribe");
        fd_envia = open(MANAGER_FIFO, O_WRONLY);
        if(fd_envia == -1){
            perror("Erro ao abrir o cliente");
            exit(1);
        }
        write(fd_envia, &msg, sizeof(msg));
    }
}
```

3.2 Manager

3.2.1 Inicialização dos recursos

- Este programa inicia a variável do ambiente “MSG_FICH”, um *array* de tópicos e um de utilizadores, uma *thread* e um *mutex* para temporizar as mensagens persistentes, uma estrutura *timeval* para temporizar o *select* e umas *variáveis auxiliares para seguranças e afins*. Por fim cria o seu *named pipe* “MANAGER_FIFO”.

```
int main(){
    char* filename = getenv("MSG_FICH");
    setbuf(stdout, NULL);
    char input[30], username[30], topico[30];
    int fd_user_envia, fd_user_recebe, size;
    fd_set readFds;
    MSG msg;
    User users[MAX_USERS];
    Topic topics[MAX_TOPICS];
    MsgToFeed msgFeed;
    struct timeval tv;
    pthread_t timer;
    pthread_mutex_t mutex;

    if (pthread_create(&timer, NULL, TimerPersistentMsg, &topics) != 0) {
        perror("Error creating timer thread");
        exit(-1);
    }

    pthread_mutex_init(&mutex, NULL);

    if (mkfifo(MANAGER_FIFO, 0660) == -1) {
        if (errno != EEXIST) {
            perror("Error creating MANAGER_FIFO");
            return -1;
        }
    }

    fd_user_recebe = open(MANAGER_FIFO, O_RDONLY | O_NONBLOCK);
    if (fd_user_recebe == -1) {
        perror("Error opening MANAGER_FIFO");
        return -1;
    }
}
```

3.2.2 Select para teclado e receção de named pipes

- Utilizamos um select para o programa receber informações do teclado e dos clientes através de *named pipes*. Além disso, inicializamos as variáveis da estrutura timeval para não haver conflitos no select.

```
while(1){
    tv.tv_sec = 5;
    tv.tv_usec = 0;
    FD_ZERO(&readFds);
    FD_SET(0, &readFds);
    FD_SET(fd_user_recebe, &readFds);

    int nfd = select(fd_user_recebe+1, &readFds, NULL, NULL, &tv);
    if (nfd < 0) {
        perror("Erro no select");
        unlink(MANAGER_FIFO);
        exit(-1);
    }
}
```

3.2.3 Comandos do Teclado

- O programa utiliza o mesmo mecanismo que o cliente para receber os comandos do teclado. No entanto, vamos analisar cada um dos comandos com algum detalhe.

```
if(FD_ISSET(0, &readFds)){
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';
}
```

- O comando “topics” mostra o conteúdo de todos os tópicos ativos como o nome, o número de mensagens persistentes e se o tópico está bloqueado. O comando “users” faz o mesmo para os utilizadores, mostra o nome e o pid de cada utilizador.

```
if (strcmp(input, "topics") == 0) {  
    printTopics(topics);  
}  
else if (strcmp(input, "users") == 0) {  
    printUsers(users);  
}
```

```
void printTopics(Topic topics[]) {  
    printf("Topics:\n");  
    for(int i = 0; i < nTopics; i++)  
        printf("- %s: %d users. %d persistent messages. blocked? (no - 0, yes - 1): %d\n", topics[i].nome,  
            topics[i].numUsers, topics[i].numMsgs, topics[i].block);  
}
```

```
void printUsers(User users[]) {  
    printf("Users: \n");  
    for(int i = 0; i < nUsers; i++) {  
        printf("- %s (PID: %d)\n", users[i].username, users[i].pid);  
    }  
}
```

- O comando “Remove” remove um utilizador e avisa todos os clientes que o mesmo foi removido. Para isso, tivemos de retirar o utilizador de todos os tópicos inscritos e removê-lo do *array* de utilizadores existente, depois enviamos um named pipe para todos os clientes ativos com o texto visando que o utilizador foi removido.

```

else if (sscanf(input, "remove %s", username) == 1) {
    for(int i = 0; i < nUsers; i++){
        if(strcmp(username, users[i].username) == 0){
            User aux;
            strcpy(aux.username, username);
            aux.pid = users[i].pid;
            sprintf(FEED_FIFO_FINAL, "FEEDFIFO%d", aux.pid);
            fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
            if(fd_user_envia == -1){
                perror("Erro ao abrir cliente\n");
                unlink(MANAGER_FIFO);
                return -1;
            }
            strcpy(msgFeed.mensagem, "Foste removido!\n");
            write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
            removeUser(&aux, topics, users);
            close(fd_user_envia);
            for(int j = 0; j < nUsers; j++){
                sprintf(FEED_FIFO_FINAL, "FEEDFIFO%d", users[j].pid);
                fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
                if(fd_user_envia == -1){
                    perror("Erro ao abrir cliente\n");
                    unlink(MANAGER_FIFO);
                    return -1;
                }
                sprintf(msgFeed.mensagem, "O utilizador %s com pid %d foi removido\n", aux.username, aux.pid);
                write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
            }
        }
    }
}

```

```

void removeUser(User *usr, Topic topics[], User users[]) {
    for (int i = 0; i < nTopics; i++) {
        for (int j = 0; j < topics[i].numUsers; j++) {
            if (topics[i].users[j].pid == usr->pid) {
                for (int k = j; k < topics[i].numUsers - 1; k++) {
                    topics[i].users[k] = topics[i].users[k + 1];
                }
                topics[i].numUsers--;
                j--;
            }
        }
    }

    for (int i = 0; i < nUsers; i++) {
        if (strcmp(users[i].username, usr->username) == 0) {
            for (int j = i; j < nUsers - 1; j++) {
                users[j] = users[j + 1];
            }
            nUsers--;
            printf("Utilizador '%s' removido com sucesso (PID: %d).\n", usr->username, usr->pid);
            break;
        }
    }
}

```

- O comando “lock” e “unlock” servem para bloquear e desbloquear um tópico específico, caso um tópico seja bloqueado, mais nenhuma mensagem pode ser passada por o mesmo, ou seja, seria necessário um “unlock” para voltar a possibilitar a troca de mensagens. Para isso, simplesmente trocamos a variável “block” para 1 e mais á frente vamos ver os ‘ifs’ que não deixam ocorrer a troca de mensagens.

```

}
else if (sscanf(input, "lock %s", topico) == 1) {
    lock(topics, topico);
}
else if (sscanf(input, "unlock %s", topico) == 1) {
    unlock(topics, topico);
}

```

```

void lock(Topic topics[], char topic[]){
    if(nTopics == 0){
        printf("Nao existem topicos!\n");
        return;
    }
    for(int i = 0; i < nTopics; i++){
        if(strcmp(topics[i].nome, topic) == 0){
            if(topics[i].block == 1){
                printf("O topico %s ja estava bloqueado!\n", topics[i].nome);
                return;
            }
            topics[i].block = 1;
            printf("Topico %s bloqueado!\n", topics[i].nome);
            return;
        }
    }

    printf("O topico %s nao existe!\n", topic);
}

void unlock(Topic topics[], char topic[]){
    if(nTopics == 0){
        printf("Nao existem topicos!\n");
        return;
    }
    for(int i = 0; i < nTopics; i++){
        if(strcmp(topics[i].nome, topic) == 0){
            if(topics[i].block == 0){
                printf("O topico %s ja estava desbloqueado!\n", topics[i].nome);
                return;
            }
            topics[i].block = 0;
            printf("Topico %s desbloqueado!\n", topics[i].nome);
            return;
        }
    }

    printf("O topico %s nao existe!\n", topic);
}

```

- O comando “show” simplesmente mostra todas as mensagens persistentes de um determinado tópico

```
else if (sscanf(input, "show %s", topico) == 1) {
    showTopic(topics, topico);
}
```

```
void showTopic(Topic topics[], char topic[]){
    for(int i = 0; i < nTopics; i++){
        if(strcmp(topics[i].nome, topic) == 0){
            if(topics[i].numMsgs == 0){
                printf("O topico %s nao tem mensagens persistentes!\n", topics[i].nome);
                return;
            }
            else{
                printf("Topico %s:\n", topics[i].nome);
                for(int j = 0; j < topics[i].numMsgs; j++){
                    printf("- %s\n", topics[i].msgPer[j].msg);
                }
                return;
            }
        }
    }
    printf("That topic does not exist\n");
}
```

- Por último, existe o comando “close”, que sai do ciclo infinito, fecha o seu *named pipe*, chama o alarme para todos os clientes fecharem, fecha a *thread* e destrói o *mutex*

```
else if (strcmp(input, "close") == 0) {
    printf("\nClosing server\n");
    break;
}
```

```
pthread_cancel(timer);
pthread_join(timer, NULL);
pthread_mutex_destroy(&mutex);

exit_signal(users);
close(fd_user_recebe);
unlink(MANAGER_FIFO);
printf("Server Closed\n");\n
```

- A função “exit_signal” envia o sinal *SIGUSR1* para todos os utilizadores

```
void exit_signal(User users[]){
    for(int i = 0; i < nUsers; i++){
        if(kill(users[i].pid, SIGUSR1) == -1){
            perror("Erro ao enviar SIGUSR1");
            unlink(FEED_FIFO_FINAL);
        }
    }
}
```

3.2.4 Comandos do Cliente

- Os comandos de cada cliente são lidos como uma estrutura MSG e dependendo do comando recebido.
- Caso o servidor receba uma estrutura cujo a mensagem e o tópico sejam NULL, quer dizer que o cliente quer dar Login, por isso, fizemos uma função para adicionar um utilizador dentro do array com todas as restrições do enunciado.

```
else if(FD_ISSET(fd_user_recebe, &readFds)){
    size = read(fd_user_recebe, &msg, sizeof(msg));
    if(size > 0){
        if(msg.msg[0] == '\0' && msg.topico[0] == '\0'){
            sprintf(FEED_FIFO_FINAL, "FEEDFIFO%d", msg.usr.pid);
            fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
            if(fd_user_envia == -1){
                perror("Erro ao abrir cliente\n");
                unlink(MANAGER_FIFO);
                return -1;
            }
            int loginRespond = (login(&msg.usr, users)) ? 1 : 0;
            write(fd_user_envia, &loginRespond, sizeof(loginRespond));
            close(fd_user_envia);
        }
    }
}
```

```

int login(User *new_user ,User users[]){
    if (nUsers >= MAX_USERS) {
        return 0;
    }
    for (int i = 0; i < nUsers; i++) {
        if (strcmp(users[i].username, new_user->username) == 0) {
            return 0;
        }
    }
    users[nUsers] = *new_user;
    nUsers++;
    printf("Utilizador '%s' logado com sucesso (PID: %d).\n", new_user->username, new_user->pid);

    return 1;
}

```

- Se receber uma estrutura cuja mensagem seja “topics”, vai enviar o mesmo texto que é visto quando se escrever “topics” pelo teclado. Para isso foi utilizado o *sprintf* para mandar todo o conteúdo dos tópicos para o cliente.

```

else if(strcmp(msg.msg, "topics")==0 && msg.topico[0] == '\0'){
    if(nTopics == 0){
        strcpy(msgFeed.mensagem, "Sistema: Nao existem topicos\n");
    }
    else{
        strcpy(msgFeed.mensagem, "");
        for(int i = 0; i < nTopics; i++){
            char temp[256];
            sprintf(temp, "- %s: %d users. %d persistent messages. blocked? (no - 0, yes - 1): %d\n",
                topics[i].nome, topics[i].numUsers, topics[i].numMsgs, topics[i].block);
            strcat(msgFeed.mensagem, temp);
        }
    }
}

```

- Se receber uma estrutura cuja mensagem seja “subscribe”, verificamos se o tópico existe e se é possível subscrever o mesmo. Depois, caso seja possível subscrever, vou enviar para o cliente todas as mensagens persistentes do tópico subscrito (se existirem).

```

else if(strcmp(msg.msg, "subscribe")==0 && msg.topico[0] != '\0'){
    int subscribeResponde = (subscribe(&msg, topics));
    sprintf(FEED_FIFO_FINAL, "FEED_FIFO%d", msg.usr.pid);
    fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
    if(fd_user_envia == -1){
        perror("Erro ao abrir cliente\n");
        return -1;
    }
    if(subscribeResponde == 0){
        strcpy(msgFeed.mensagem, "Sistema: Ja estas inscrito nesse topico ou esse topico nao existe!\n");
    }
    else{
        for(int j = 0; j < nTopics; j++){
            if(strcmp(topics[j].nome, msg.topico) == 0){
                for(int jj = 0; jj < topics[j].numMsgs; jj++){
                    sprintf(msgFeed.mensagem, "[%s] %s: %s\n", topics[j].msgPer[jj].topico, topics[j].msgPer[jj].usr.username, topics[j].msgPer[jj].msg);
                    write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
                }
            }
        }
        sprintf(msgFeed.mensagem, "Sistema: Inscrito com sucesso em %s\n", msg.topico);
    }
    write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
    close(fd_user_envia);
}

```

```

int subscribe(MSG *msg, Topic topics[]){
    for(int i = 0; i < nTopics; i++){
        if(strcmp(msg->topico, topics[i].nome) == 0){
            for(int j = 0; j < topics[i].numUsers; j++){
                if(msg->usr.pid == topics[i].users[j].pid)
                    return 0;
            }
            int aux = topics[i].numUsers;
            topics[i].users[aux] = msg->usr;
            topics[i].numUsers++;
            printf("O utilizador com pid %d subscreveu em %s\n", msg->usr.pid, topics[i].nome);
            return 1;
        }
    }
    return 0;
}

```

- Se receber uma estrutura cuja mensagem seja “exit”, vamos remover o utilizador com a mesma função mostrada em cima. Neste caso, nenhum cliente vai saber que o utilizador saiu.

```

else if(strcmp(msg.msg, "exit") == 0 && msg.topico[0] != '\0'){
    removeUser(&msg.usr, topics, users);
}

```

- Se receber uma estrutura cuja mensagem seja “unsubscribe”, verificamos se o tópico existe e se é possível tirar a subscrição do mesmo. Se for possível, verificamos se o tópico tem algum cliente ou alguma mensagem persistente, se o valor for 0 em ambos os casos, removemos o tópico com a função “removeTopic”.

```
else if(strcmp(msg.msg, "unsubscribe")==0 && msg.topico[0] != '\0'){
    int unsubscribeResponde = (unsubscribe(&msg, topics));
    sprintf(FEED_FIFO_FINAL, "FEED_FIFO%d", msg.usr.pid);
    fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
    if(fd_user_envia == -1){
        perror("Erro ao abrir cliente\n");
        return -1;
    }
    if(unsubscribeResponde == 0){
        strcpy(msgFeed.mensagem, "Sistema: Nao estas inscrito nesse topico ou esse topico nao existe!\n");
    }
    else{
        sprintf(msgFeed.mensagem, "Sistema: Desinscrito com sucesso em %s\n", msg.topico);
    }
    write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
    close(fd_user_envia);
}
```

```
int unsubscribe(MSG *msg, Topic topics[]) {
    for (int i = 0; i < nTopics; i++) {
        if (strcmp(msg->topico, topics[i].nome) == 0) {
            for (int j = 0; j < topics[i].numUsers; j++) {
                if (msg->usr.pid == topics[i].users[j].pid) {
                    for (int k = j; k < topics[i].numUsers - 1; k++) {
                        topics[i].users[k] = topics[i].users[k + 1];
                    }
                    topics[i].numUsers--;
                    printf("O utilizador com pid %d cancelou a subscrição em %s\n", msg->usr.pid, topics[i].nome);
                    if(topics[i].numUsers == 0 && topics[i].numMsgs == 0)
                        removeTopic(&topics[i], topics);
                    return 1;
                }
            }
        }
    }
    return 0;
}
```

```
void removeTopic(Topic *topic, Topic topics[]){
    for(int i = 0; i < nTopics; i++){
        if(strcmp(topics[i].nome, topics->nome) == 0){
            for(int j = 1; j < nTopics - 1; j++){
                topics[j] = topics[j + 1];
            }
            nTopics--;
            printf("O topico %s foi apagado por estar inativo\n", topics[i].nome);
            break;
        }
    }
}
```

- Por último e não menos importante, se o servidor receber alguma mensagem que não seja as que estão em cima e se o nome do tópico não for NULL, criamos um tópico novo se ainda não existir. Caso exista, partilhamos com todos os clientes inscritos (caso não esteja bloqueado)

```

else if(msg.msg[0] != '\0' && msg.topico[0] != '\0'){
    int topicResponde = (msgManager(msg, topics));
    sprintf(FEED_FIFO_FINAL, "FEED_FIFO%d", msg.usr.pid);
    fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
    if(fd_user_envia == -1){
        perror("Erro ao abrir cliente\n");
        return -1;
    }
    if(topicResponde == 1){
        topics[nTopics-1].m = &mutex;
        sprintf(msgFeed.mensagem, "[%s] %s: %s\n", msg.topico, msg.usr.username, msg.msg);
        write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
        close(fd_user_envia);
    }
    else if(topicResponde == 0){
        strcpy(msgFeed.mensagem, "Sistema: Limite de topicos atingido ou topico ja existe!\n");
        write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
        close(fd_user_envia);
    }
    else if(topicResponde == -1){
        strcpy(msgFeed.mensagem, "Sistema: O topico esta bloqueado, tente mais tarde\n");
        write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem));
        close(fd_user_envia);
    }
    else{
        close(fd_user_envia);
        sprintf(msgFeed.mensagem, "[%s] %s: %s\n", msg.topico, msg.usr.username, msg.msg);
        for(int i = 0; i < nTopics; i++){
            if(strcmp(topics[i].nome, msg.topico) == 0){
                for(int j = 0; j < topics[i].numUsers; j++){
                    sprintf(FEED_FIFO_FINAL, "FEED_FIFO%d", topics[i].users[j].pid);

                    fd_user_envia = open(FEED_FIFO_FINAL, O_WRONLY);
                    if (fd_user_envia == -1) {
                        perror("Erro ao abrir cliente\n");
                        unlink(MANAGER_FIFO);
                        exit(1);
                    }

                    if (write(fd_user_envia, &msgFeed.mensagem, sizeof(msgFeed.mensagem)) == -1) {
                        perror("Erro ao enviar mensagem\n");
                    }

                    close(fd_user_envia);
                }
            }
        }
    }
}

```

- Dependendo do *return* da função em baixo, o main vai realizar uma destas operações: Ver se o tópico existe e o cliente não está inscrito ou se o limite foi atingido; Ver se o tópico está bloqueado; Mandar a mensagem para todos os utilizadores inscritos no tópico.

```

int msgManager(MSG msg, Topic topics[]) {
    for (int i = 0; i < nTopics; i++) {
        if (strcmp(topics[i].nome, msg.topico) == 0) {
            for (int j = 0; j < topics[i].numUsers; j++) {
                if (msg.usr.pid == topics[i].users[j].pid) {
                    if (topics[i].block) {
                        return -1;
                    }
                    if(msg.dur > 0){
                        topics[i].numMsgs++;
                        int aux = topics[i].numMsgs;
                        topics[i].msgPer[aux-1] = msg;
                    }
                    return 2;
                }
            }
            return 0;
        }
    }

    if (nTopics >= MAX_TOPICS) {
        printf("Limite de tópicos atingido!\n");
        return 0;
    }

    Topic new_topic;
    strcpy(new_topic.nome, msg.topico);
    new_topic.users[0] = msg.usr;
    new_topic.block = 0;
    if(msg.dur > 0){
        new_topic.numMsgs = 1;
        new_topic.msgPer[0] = msg;
    }
    else new_topic.numMsgs = 0;
    new_topic.numUsers = 1;

    topics[nTopics] = new_topic;
    nTopics++;

    printf("Tópico %s criado com sucesso e utilizador com pid %d subscrito.\n", new_topic.nome, msg.usr.pid);

    return 1;
}

```

- Esta função primeiro vê se o tópico já existe. Se o tópico existir, a função vai ver se o cliente está ou não inscrito. Se o tópico não existir, verifica se o máximo foi atingido e se não foi atingido, cria um novo tópico.

3.2.5 Guardar e Abrir Ficheiro

- Quando o servidor inicia, o ficheiro é logo aberto com a função “load_persistent_messages”.

```
if (filename) {
    load_persistent_messages(filename, topics);
} else {
    printf("Variável de ambiente MSG_FICH não definida.\n");
}
```

```
void load_persistent_messages(const char* filename, Topic topics[]) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        perror("Erro ao abrir ficheiro para leitura");
        return;
    }

    char line[512];
    while (fgets(line, sizeof(line), file)) {
        char usernameAux[30], topicAux[MAX_TOPIC_SIZE], msgAux[MAX_MSG_SIZE];
        int dur;
        if (sscanf(line, "%s %s %d %[^\\n]", topicAux, usernameAux, &dur, msgAux) == 4) {
            MSG msg;
            msg.dur = dur;
            strcpy(msg.msg, msgAux);
            strcpy(msg.topico, topicAux);
            strcpy(msg.usr.username, usernameAux);
            for(int i = 0; i < nTopics; i++){
                if(strcmp(topics[i].nome, topicAux) == 0){
                    if(!addPersistentMsg(msg, topics)){
                        printf("Error creating persistent messages\n");
                        unlink(MANAGER_FIFO);
                        exit(-1);
                    }
                }
            }

            Topic new_topic;
            strcpy(new_topic.nome, topicAux);
            new_topic.block = 0;
            new_topic.numMsgs = 1;
            new_topic.msgPer[0] = msg;
            new_topic.numUsers = 0;
            topics[nTopics] = new_topic;
            nTopics++;
        }
    }

    fclose(file);
    printf("Mensagens persistentes carregadas de '%s'\n", filename);
}
```

- Esta função lê linha por linha e cria uma nova mensagem para cada linha, se o tópico já existe (de uma linha anterior por exemplo), adiciona uma mensagem persistente ao mesmo e para isso criamos uma pequena função chamada “*addPersistentMsg*” que adiciona a mensagem ao tópico. Caso o tópico não exista, cria um novo.

```
int addPersistentMsg(MSG msg, Topic topics[]){
    for(int i = 0; i < nTopics; i++){
        if(strcmp(msg.topico, topics[i].nome) == 0){
            if(topics[i].numMsgs >= MAX_MSG_PERSISTENTE)
                return 0;
            int aux = topics[i].numMsgs;
            topics[i].msgPer[aux] = msg;
            topics[i].numMsgs++;
            printf("Message added to topic %s for %d seconds", topics[i].nome, msg.dur);
            return 1;
        }
    }
    return 0;
}
```

- Quando o servidor desligar, chamados a função “*save_persistent_messages*” que salva todas as mensagens persistentes por linha.

```
void save_persistent_messages(const char* filename, Topic topics[]) {
    if (!filename) {
        fprintf(stderr, "Erro: A variável de ambiente MSG_FICH não está definida.\n");
        return;
    }
    FILE* file = fopen(filename, "w");
    if (!file) {
        perror("Erro ao abrir ficheiro para escrita");
        return;
    }

    for (int i = 0; i < nTopics; i++) {
        for (int j = 0; j < topics[i].numMsgs; j++) {
            MSG* msg = &topics[i].msgPer[j];
            if (msg->dur > 0) {
                fprintf(file, "%s %s %d %s\n",
                        topics[i].nome,
                        msg->usr.username,
                        msg->dur,
                        msg->msg);
            }
        }
    }

    fclose(file);
    printf("Mensagens persistentes salvas em '%s'\n", filename);
}
```

- Esta função cria uma mensagem temporária e utiliza o *fprintf* para guardar o conteúdo requerido por linha no ficheiro.

3.3 Temporizador das Mensagens Persistentes

- Utilizamos uma thread para temporizar todas as mensagens persistentes. Na thread, por cada segundo, verificamos se cada tópico contém pelo menos uma mensagem persistente. Se tiver, iniciamos o mutex para reduzir conflitos, reduzimos o inteiro “dur” por 1 para simular que passou 1 segundo. Caso o inteiro fique 0, retiramos a mensagem das mensagens persistentes e se o tópico não tiver nenhum utilizador inscrito ou nenhuma mensagem persistente, utilizamos a mesma função que utilizamos em cima para remover o tópico.

```
void* TimerPersistentMsg(void* dados) {
    Topic* topics = (Topic*) dados;

    while (1) {
        sleep(1);

        for (int i = 0; i < nTopics; i++) {
            Topic* topic = &topics[i];

            if (topic->numMsgs <= 0) {
                continue;
            }

            pthread_mutex_lock(topic->m);

            for (int j = 0; j < topic->numMsgs; j++) {
                MSG* message = &topic->msgPer[j];
                if (message->dur > 0) {
                    message->dur--;
                    if (message->dur == 0) {
                        printf("Mensagem do tópico '%s' removida: %s\n", topic->nome, message->msg);
                        for (int k = j; k < topic->numMsgs - 1; k++) {
                            topic->msgPer[k] = topic->msgPer[k + 1];
                        }
                        topic->numMsgs--;
                        --j;
                        if (topic->numMsgs == 0 && topic->numUsers == 0)
                            removeTopic(&topics[i], topics);
                    }
                }
            }

            pthread_mutex_unlock(topic->m);
        }
    }

    return NULL;
}
```

4. Makefile

4.1 Estrutura

- As variáveis definem o compilador (gcc) , flags de compilação (wall e -g) para ver os warnings e os caminhos dos arquivos e diretorias.

```
CC = gcc
CFLAGS = -Wall -g
LDFLAGS =

SRC_DIR = src
OBJ_DIR = obj
BIN_DIR = bin
```

4.2 Regras

- all: Compila os dois programas (Feed e Manager).
- Clean: Remove arquivos gerados durante a compilação.
- Diretorias \$(OBJ_DIR) e \$(BIN_DIR): Criadas automaticamente se não existirem.

```
all: $(FEED) $(BROKER)

$(FEED): $(FEED_OBJ)
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

$(BROKER): $(BROKER_OBJ)
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

$(OBJ_DIR)/Feed.o: $(FEED_SRC)
    $(CC) $(CFLAGS) -c -o $@ $<

$(OBJ_DIR)/Manager.o: $(BROKER_SRC)
    $(CC) $(CFLAGS) -c -o $@ $<

clean:
    rm -rf $(OBJ_DIR)/*.o $(BIN_DIR)/*

$(OBJ_DIR) $(BIN_DIR):
    mkdir -p $@
```

4.3 Compilação

- Cada ficheiro .c é compilado para um ficheiro .o e, posteriormente, ligado para gerar os executáveis no diretório bin.

```
FEED = $(BIN_DIR)/Feed
BROKER = $(BIN_DIR)/Manager

FEED_SRC = $(SRC_DIR)/Feed.c
BROKER_SRC = $(SRC_DIR)/Manager.c

FEED_OBJ = $(OBJ_DIR)/Feed.o
BROKER_OBJ = $(OBJ_DIR)/Manager.o
```

5. Conclusão

O trabalho cumpriu com os requisitos especificados no enunciado, oferecendo uma plataforma funcional e alinhada com os princípios de programação e arquitetura de sistemas UNIX. Todas as funcionalidades foram implementadas e verificadas em cenários de teste representativos, garantindo tanto a estabilidade como a eficiência da solução.

Além disso, a implementação destacou-se pela utilização correta dos recursos do sistema operativo e pelo cumprimento das boas práticas de programação. A documentação clara facilita a compreensão do código. Este projeto permitiu consolidar conhecimentos práticos recebidos das aulas sobre sistemas operativos, comunicação entre processos e gestão de recursos em ambientes UNIX.

FIM